

Assignment 4

Code

```
# ----- Settings -----
# upright [0, 0]
GOAL_ANGLE = 0.0

# horizontal [pi/2, 0]
# GOAL_ANGLE = onp.pi / 2.0

# horizontal [-pi/2, 0]
# GOAL_ANGLE = -onp.pi / 2.0
# -----

def run_lqr_control():

    # setup environment
    x_initial = jnp.array([-onp.pi, 0.0])
    env = JaxPendulum(initial_state=x_initial, render_mode="human")
    x0, _ = env.reset()

    # define goal
    x_goal = jnp.array([GOAL_ANGLE, 0.0])
    # x_goal = jnp.array([0, 0.1])

    # linearize around the goal
    G, H, c = linearise_autodiff(env.F_step, x_goal)

    # ----- tune Q and R -----
    # Q: > penalize angle error heavily
    #     > penalize angular velocity error moderately
    Q = onp.diag([10.0, 1.0]) # [angle-error, angle-velocity]
    # R: > penalize control effort. lower R = more aggressive controller
    R = onp.array([[0.1]]) # control-effort
    # -----

    C = onp.zeros_like(G)
    D = onp.zeros_like(H)

    # solve DARE to get k
    ss = control.ss(G, H, C, D, dt=env.dt)
    K, S, E = control.lqr(ss, Q, R)

    # ----- compensation for c -----
    # at angle=0, c is approx 0
```

```

# at angle=pi/2, c is non-zero (gravity)
# we need a control u_offset such that: h @ u_offset + c = 0
# as suggested we use the pseudo-inverse of H to isolate u_offset:
u_offset = -jnp.linalg.pinv(H) @ c
# -----

# print statistics
print(f"Goal: {x_goal}")
print(f"K: {K}")
print(f"u_offset: {u_offset}")

# simulation loop
x_k = x0
trajectory = [x0]
u_history = []
cost_history = []

for k in range(100):
    env.render()

    # calculate error
    error = x_k - x_goal

    u_k = -K @ error + u_offset

    # keep control in valid range
    u_k = jnp.clip(u_k, env.action_space.low, env.action_space.high)

    # take a step in the environment
    x_kp1, _, _, _ = env.step(u_k)

    # store data
    trajectory.append(x_kp1)
    u_history.append(u_k)

    # calculate quadratic cost for plotting
    cost = error.T @ Q @ error + u_k.T @ R @ u_k
    cost_history.append(cost)

    x_k = x_kp1
    time.sleep(0.05)

# plot results
trajectory = onp.array(trajectory)
u_history = onp.array(u_history).flatten()

plt.figure(figsize=(12, 4))

plt.subplot(1, 3, 1)
plt.plot(trajectory[:, 0], label="angle")
plt.plot(trajectory[:, 1], label="velocity")
plt.axhline(x_goal[0], color='r', linestyle='--', label="goal angle")
plt.title("state trajectories")

```

```

plt.legend()
plt.grid()

plt.subplot(1, 3, 2)
plt.plot(u_history)
plt.title("control input (u)")
plt.grid()

plt.subplot(1, 3, 3)
plt.plot(cost_history)
plt.title("quadratic cost")
plt.grid()

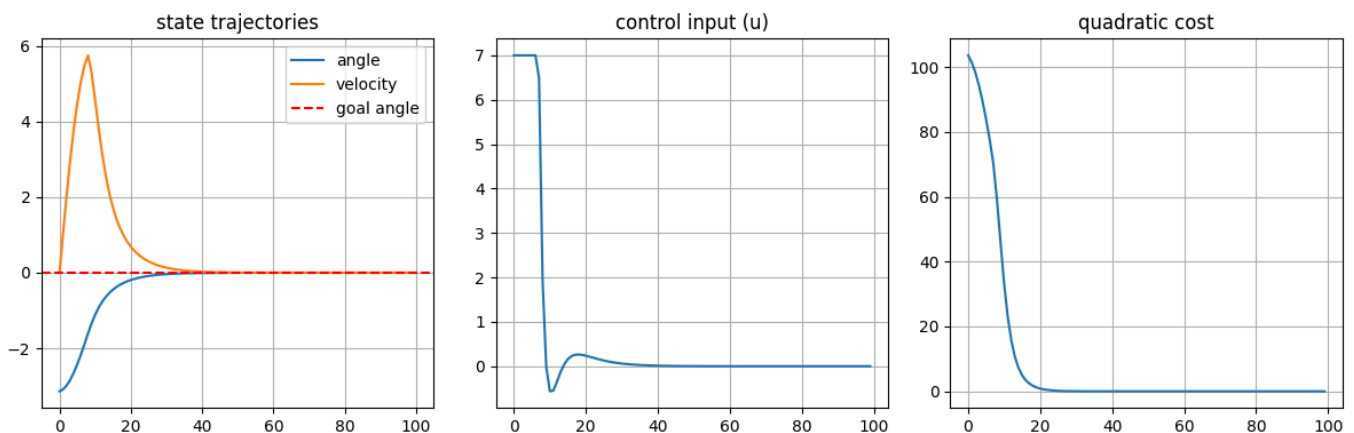
plt.tight_layout()
plt.show()

if __name__ == "__main__":
    run_lqr_control()

```

(1)

- the constant factor c is approximately 0
 - as gravity doesn't influence pendulum at goal-state (upright)
- so we don't need any c -compensation here
- see 0-0-no_c.gif



(2)

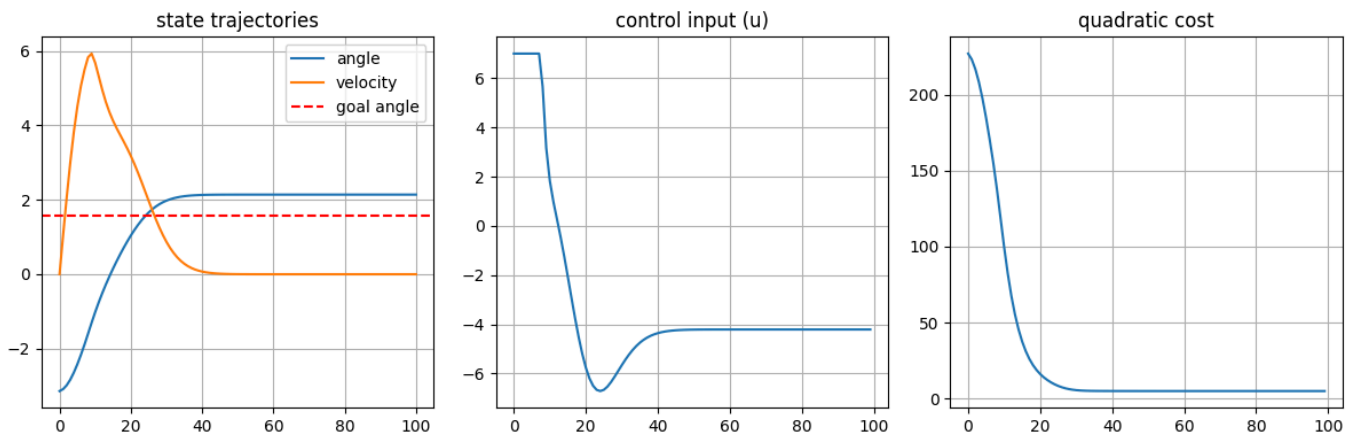
- at angles $\frac{\pi}{2}$ and $-\frac{\pi}{2}$, the pendulum is in a horizontal position
- at this position gravity pulls the pendulum down
- the term c represents this gravitational acceleration

- if we use pure LQR, at the (horizontal) goal state the error is zero
- and therefore the controller assumes that no input is required anymore
- so the controller turns off
- but gravity still pulls the pendulum down
- therefore the pendulum will drop down below the horizontal target state
- the controller activates again and fights back until it generates enough input to fight gravity
- the pendulum ends up stopping at a state that is not the target state...
- **Compensation:**
 - therefore we need to calc the exact force that is needed to fight gravity at the target-state:
 - state-space equation: $x_{k+1} = Gx_k + Hu_k + c$
 - at the target state: $x_t = x_k + x_{k+1}$
 - state-space equation at the target state: $x_t = Gx_t + Hu_t + c$
 - isolate u_t : $Hu_t = x_t - Gx_t - c$
 - we assume that: $x_t = 0$
 - therefore: $Hu_t = -c$
 - we need to use pseudo inverse: $u_t = -\hat{H}c$
 - in the code we called u_t : `u_offset`

GOAL_ANGLE = $\frac{\pi}{2}$

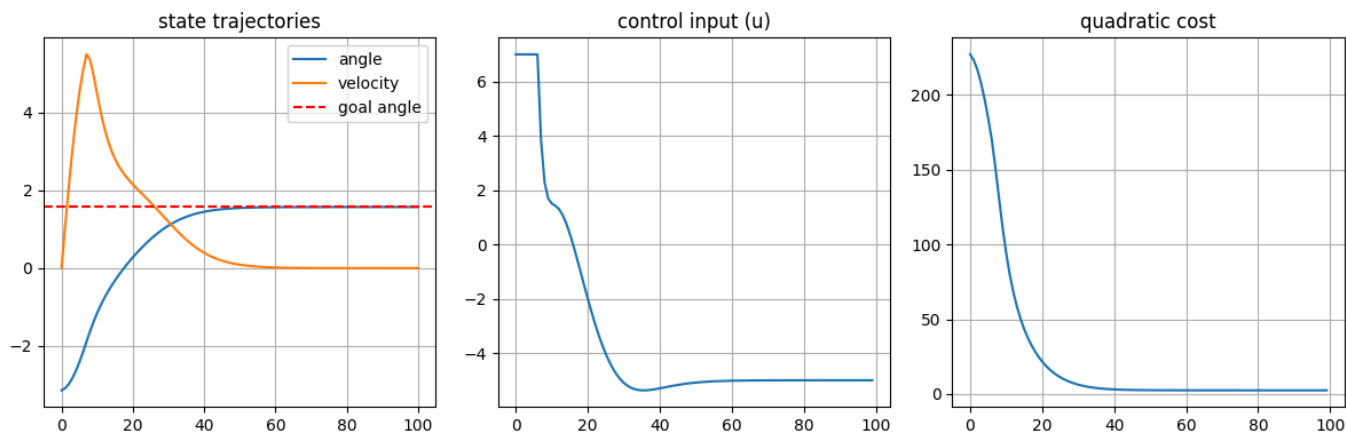
WITHOUT Compensation

- see pi-0-no_c.gif



WITH Compensation

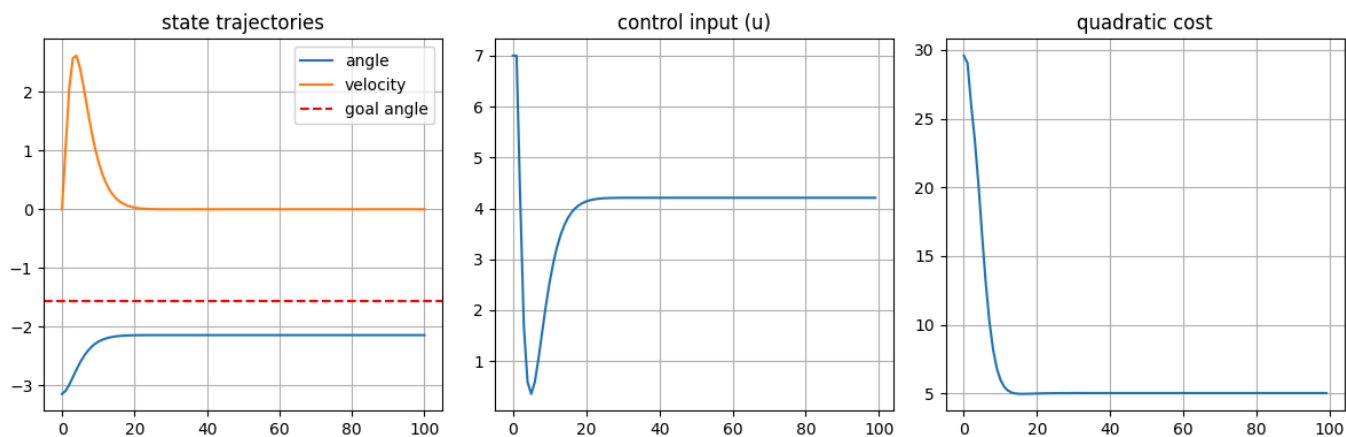
- see pi-0-with_c.gif



$$\text{GOAL_ANGLE} = -\frac{\pi}{2}$$

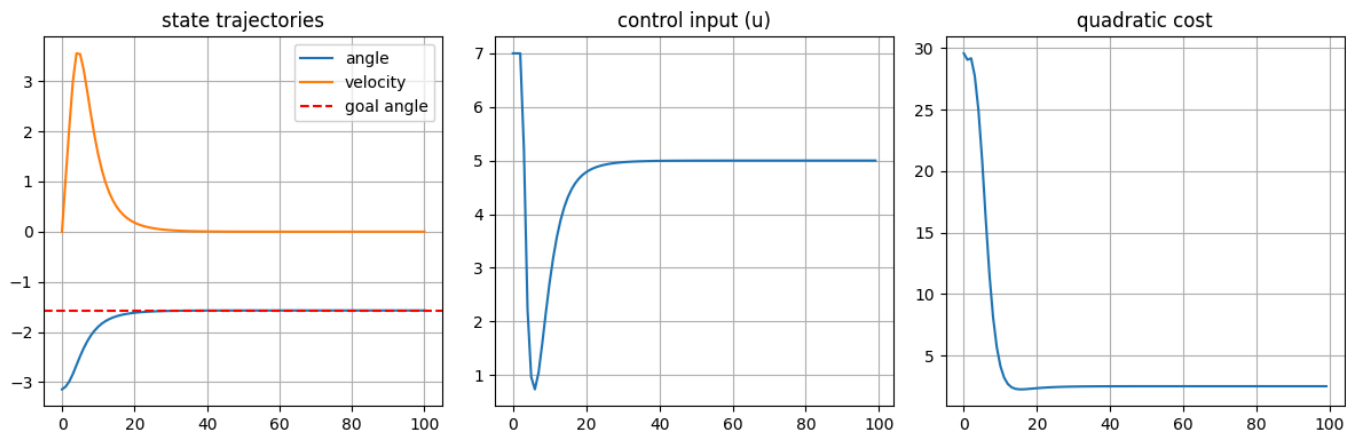
WITHOUT Compensation

- see -pi-0-no_c.gif



WITH Compensation

- see -pi-0-with_c.gif



(3)

GOAL_STATE=[0, 1.0]

- at the goal-state, the pendulum should have angle=0, but should move with velocity=1
- this is impossible for the pendulum
- the controller finds a compromise where the angle-error and velocity-error are balanced against the control-cost (u) and then it stays at that position
- see 0-1.gif

